

BỘ GIÁO DỤC & ĐÀO TẠO
ĐẠI HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
KHOA KINH TẾ



BÁO CÁO THỰC HÀNH LAB 03
ĐỀ TÀI: CÁC GIẢI THUẬT PHÂN CỤM CƠ BẢN
(K-MEANS VÀ PHÂN CỤM ĐA CẤP)

MÔN HỌC: PHÂN TÍCH VÀ QUẢN LÝ ĐẦU TƯ NÂNG CAO

Giảng viên hướng dẫn: TS. Đỗ Như Tài
LHP: 26D2ECO50118301

Thành viên nhóm:

Võ Duy Huy Hoàng
Nguyễn Hữu Khoa
Hoàng Thị Hường

THÔNG TIN THÀNH VIÊN

Họ và tên	Mã số sinh viên	Tỷ trọng đóng góp
Võ Duy Huy Hoàng	87243020093	100%
Hoàng Thị Hường	88242020100	100%
Nguyễn Hữu Khoa	88234020092	100%

MỤC LỤC

THÔNG TIN THÀNH VIÊN.....	2
MỤC LỤC.....	3
A. MỤC TIÊU.....	4
B. KẾT CẤU.....	4
C. NỘI DUNG.....	4
3.1. GIẢI THUẬT K-MEANS.....	4
3.1.1. Ôn tập lý thuyết.....	4
3.1.3. Bài tập thực hành 1: Phân cụm K-Means trên tập dữ liệu Penguins.....	7
Bước 1: Nạp và tiền xử lý dữ liệu.....	7
Bước 2: Xác định số cụm tối ưu (Elbow Method & Silhouette Score).....	8
Bước 3: Xây dựng mô hình K-Means với $k=3$	8
3.1.4. Bài tập thực hành 2: Phân cụm K-Means trên tập dữ liệu Online Retail..	10
Bước 1: Làm sạch dữ liệu giao dịch.....	10
Bước 2: Xây dựng bảng RFM cho từng khách hàng.....	10
Bước 3: Chuẩn hóa và xác định số cụm tối ưu.....	11
Bước 4: Xây dựng mô hình và trực quan hóa.....	11
3.2. GIẢI THUẬT PHÂN CỤM ĐA CẤP.....	13
3.2.1. Ôn tập lý thuyết.....	13
3.2.3. Bài tập thực hành 1: Phân cụm đa cấp trên tập dữ liệu Penguins.....	15
Bước 1: Vẽ dendrogram với Ward linkage.....	15
Bước 2: So sánh các phương pháp linkage.....	16
Bước 3: Xây dựng mô hình cuối cùng và trực quan hóa.....	17
Bước 4: So sánh với K-Means bằng Silhouette Score.....	18
3.2.4. Bài tập thực hành 2: Phân cụm đa cấp trên tập dữ liệu Online Retail.....	18
Bước 1: Vẽ dendrogram trên mẫu khách hàng.....	18
Bước 2: So sánh các phương pháp linkage ($k=4$).....	19
Bước 3: Xây dựng mô hình cuối cùng (Ward, $k=4$) và trực quan hóa.....	20
Bước 4: So sánh với K-Means bằng Silhouette Score.....	20
D. TÓM TẮT.....	21

A. MỤC TIÊU

- Hướng dẫn thực hiện phân cụm dữ liệu với giải thuật K-Means.
- Hướng dẫn thực hiện phân cụm đa cấp.
- Hướng dẫn ôn tập các kiến thức lý thuyết về K-Means và phân cụm đa cấp.

B. KẾT CẤU

Nội dung báo cáo bao gồm 2 phần:

- Giải thuật K-Means
- Giải thuật phân cụm đa cấp

C. NỘI DUNG

3.1. GIẢI THUẬT K-MEANS

3.1.1. Ôn tập lý thuyết

1. Giải thuật K-Means hoạt động như thế nào? Hãy giải thích các bước chính trong quy trình phân cụm.

K-Means là giải thuật phân cụm dựa trên khoảng cách (thường là khoảng cách Euclidean), nhằm chia tập dữ liệu thành K cụm sao cho các điểm trong cùng một cụm có độ tương đồng cao và khác biệt nhiều so với các điểm ở cụm khác. Quy trình gồm các bước chính:

- 1) Khởi tạo: chọn ngẫu nhiên K điểm làm tâm cụm (centroid) ban đầu.
- 2) Gán nhãn: mỗi điểm dữ liệu được gán vào cụm có tâm gần nó nhất (theo khoảng cách Euclidean).
- 3) Cập nhật tâm cụm: tính lại tâm mới của mỗi cụm bằng trung bình cộng của tất cả các điểm thuộc cụm đó.
- 4) Lặp lại bước 2 và 3 cho đến khi các tâm cụm không còn thay đổi đáng kể (hội tụ) hoặc đạt số lần lặp tối đa.

2. Tại sao cần chọn số lượng cụm (K) trước khi chạy K-Means? Làm thế nào để xác định giá trị K tối ưu?

K-Means yêu cầu biết trước số cụm K vì thuật toán khởi tạo K tâm cụm ngay từ đầu và toàn bộ quá trình lặp chỉ tối ưu vị trí của K tâm này - thuật toán không tự sinh hay loại bỏ cụm trong quá trình chạy.

Để tìm K tối ưu, phổ biến nhất là hai phương pháp: (1) Phương pháp Elbow (khủy tay): chạy K-Means với nhiều giá trị K, vẽ đồ thị Inertia (tổng bình phương khoảng cách trong cụm - WCSS) theo K, chọn điểm mà tốc độ giảm của Inertia chậm lại rõ rệt (tạo hình khủy tay). (2) Silhouette Score: đo mức độ một điểm phù hợp với cụm của nó so với các cụm khác, chọn K

cho điểm Silhouette trung bình cao nhất. Ngoài ra còn có Gap Statistic hoặc kiến thức nghiệp vụ (domain knowledge) để hỗ trợ lựa chọn K.

3. Hàm mục tiêu (objective function) của K-Means là gì? Nó đo lường điều gì trong quá trình phân cụm?

Hàm mục tiêu của K-Means là tổng bình phương khoảng cách từ mỗi điểm dữ liệu đến tâm cụm mà nó thuộc về, gọi là WCSS (Within-Cluster Sum of Squares) hay Inertia:

$$J = \sum_{k=1..K} \sum_{x \text{ thuộc cụm } k} \|x - \mu_k\|^2$$

trong đó μ_k là tâm của cụm k. K-Means tìm cách gán nhãn và cập nhật tâm cụm sao cho giá trị J này nhỏ nhất có thể, tức là làm cho các điểm trong cùng một cụm càng gần tâm cụm (càng "chặt") càng tốt. Đây là bài toán tối ưu không lồi nên K-Means chỉ đảm bảo hội tụ về cực tiểu cục bộ, không đảm bảo tìm được cực tiểu toàn cục.

4. Những hạn chế của K-Means là gì? Trong trường hợp nào K-Means có thể cho kết quả không tốt?

Một số hạn chế chính của K-Means:

- Phải chọn trước số cụm K, trong khi số cụm thực tế thường không biết trước.
- Nhạy cảm với vị trí khởi tạo tâm cụm ban đầu, có thể hội tụ về cực tiểu cục bộ khác nhau giữa các lần chạy.
- Giả định các cụm có dạng hình cầu (convex), kích thước và mật độ tương đồng nhau; hoạt động kém khi cụm có hình dạng phức tạp, không lồi hoặc kích thước/mật độ chênh lệch lớn.
- Nhạy cảm với outlier (giá trị ngoại lai) vì dùng trung bình cộng để tính tâm cụm.
- Chỉ áp dụng tốt cho dữ liệu số (numeric) và khoảng cách Euclidean; khó áp dụng trực tiếp cho dữ liệu phân loại (categorical).
- Cần chuẩn hóa dữ liệu (scaling) trước khi phân cụm, nếu không các đặc trưng có thang đo lớn sẽ chi phối kết quả.

5. Viết đoạn code mẫu bằng Python (sử dụng Scikit-learn) để triển khai K-Means Clustering. Mô tả các bước thực hiện.

Các bước thực hiện: (1) nạp và tiền xử lý dữ liệu, (2) chuẩn hóa đặc trưng, (3) khởi tạo và huấn luyện mô hình KMeans, (4) gán nhãn cụm cho từng mẫu dữ liệu.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 1. Nạp dữ liệu
df = pd.read_csv("data.csv")
X = df[["feature_1", "feature_2", "feature_3"]]
```

```
# 2. Chuan hoa du lieu
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Khoi tao va huan luyen mo hinh K-Means
kmeans = KMeans(n_clusters=3, init="k-means++",
                 n_init=10, random_state=42)
kmeans.fit(X_scaled)

# 4. Gan nhan cum cho tung mau du lieu
df["Cluster"] = kmeans.labels_
print(df.head())
```

6. Sử dụng phương pháp nào trong Python để chọn số cụm K tối ưu (Elbow Method, Silhouette Score)? Chia sẻ đoạn code mẫu.

Có thể dùng vòng lặp thử nhiều giá trị K, lưu lại Inertia (cho Elbow Method) và Silhouette Score, sau đó trực quan hóa để chọn K phù hợp.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

inertias, scores = [], []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, init="k-means++",
                 n_init=10, random_state=42).fit(X_scaled)
    inertias.append(km.inertia_)
    scores.append(silhouette_score(X_scaled, km.labels_))

# Bieu do Elbow
plt.plot(K_range, inertias, marker="o")
plt.xlabel("So cum K"); plt.ylabel("Inertia"); plt.show()

# Bieu do Silhouette
plt.plot(K_range, scores, marker="o")
plt.xlabel("So cum K"); plt.ylabel("Silhouette Score"); plt.show()
```

7. K-Means nhạy cảm với giá trị khởi tạo (initial centroids), bạn sẽ làm gì để đảm bảo kết quả ổn định (ví dụ: K-Means++)?

Để giảm ảnh hưởng của việc khởi tạo ngẫu nhiên, có thể áp dụng các biện pháp sau:

- Sử dụng K-Means++ (tham số `init="k-means++"` trong Scikit-learn): thay vì chọn tâm cụm hoàn toàn ngẫu nhiên, K-Means++ chọn các tâm cụm ban đầu cách xa nhau, giúp thuật toán hội tụ nhanh hơn và ổn định hơn.
- Chạy thuật toán nhiều lần với các khởi tạo khác nhau (tham số `n_init`), sau đó chọn kết quả có Inertia thấp nhất - đây cũng là hành vi mặc định của KMeans trong Scikit-learn.

- Cố định `random_state` để đảm bảo khả năng tái lập kết quả (reproducibility) khi cần so sánh giữa các lần chạy.

8. Làm thế nào để đánh giá chất lượng của các cụm được tạo bởi K-Means? Bạn sử dụng chỉ số nào (Silhouette Score, WCSS)?

Một số chỉ số thường dùng để đánh giá chất lượng phân cụm:

- Within-Cluster Sum of Squares (WCSS/Inertia): tổng bình phương khoảng cách từ các điểm đến tâm cụm của chúng; giá trị càng nhỏ thì các cụm càng "chặt", nhưng luôn giảm khi K tăng nên chỉ dùng để so sánh tương đối (Elbow Method), không dùng để chọn K lớn nhất.

- Silhouette Score: đo mức độ tách biệt giữa các cụm, nhận giá trị từ -1 đến 1; giá trị càng gần 1 cho thấy các điểm được gán đúng cụm và các cụm tách biệt rõ ràng.

- Ngoài ra có thể dùng Davies-Bouldin Index (càng nhỏ càng tốt) hoặc Calinski-Harabasz Index (càng lớn càng tốt) để đánh giá bổ sung, cùng với việc trực quan hóa cụm (scatter plot, pairplot) để kiểm tra định tính.

3.1.3. Bài tập thực hành 1: Phân cụm K-Means trên tập dữ liệu Penguins

Đề bài: Xây dựng mô hình phân cụm K-Means trên tập dữ liệu chim cánh cụt (Penguins). Bộ dữ liệu Palmer Penguins gồm 344 quan sát với các đặc trưng: chiều dài mỏ (`bill_length_mm`), độ sâu mỏ (`bill_depth_mm`), chiều dài cánh (`flipper_length_mm`), khối lượng cơ thể (`body_mass_g`), loài (`species`), đảo (`island`) và giới tính (`sex`).

Dữ liệu lấy tại:

<https://www.kaggle.com/code/youssefaboelwafa/clustering-penguins-species-k-means-clustering>

Bước 1: Nạp và tiền xử lý dữ liệu

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

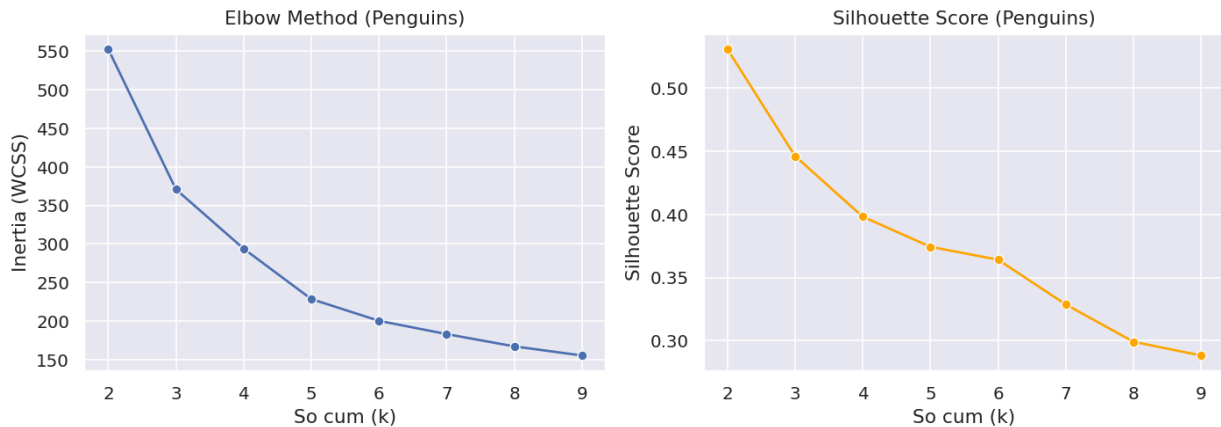
df = pd.read_csv("penguins.csv")
df = df.dropna().reset_index(drop=True) # 344 -> 333 dòng

num_cols = ["bill_length_mm", "bill_depth_mm",
            "flipper_length_mm", "body_mass_g"]
X = StandardScaler().fit_transform(df[num_cols])
```

Sau khi loại bỏ các dòng thiếu dữ liệu, tập dữ liệu còn lại 333 quan sát với 4 đặc trưng số được sử dụng để phân cụm. Dữ liệu được chuẩn hóa (`StandardScaler`) trước khi đưa vào K-Means vì các đặc trưng có thang đo rất khác nhau (mm so với gram).

Bước 2: Xác định số cụm tối ưu (Elbow Method & Silhouette Score)

```
inertias, scores = [], []
for k in range(2, 10):
    km = KMeans(n_clusters=k, init="k-means++",
                n_init=10, random_state=42).fit(X)
    inertias.append(km.inertia_)
    scores.append(silhouette_score(X, km.labels_))
```



Hình 1 - Biểu đồ Elbow và Silhouette Score cho tập dữ liệu Penguins

Nhận xét: đường Inertia giảm mạnh từ k=2 đến k=4 rồi giảm chậm dần, cho thấy khuỷu tay nằm quanh k=3-4. Silhouette Score đạt cao nhất tại k=2 (0.5308) nhưng giảm dần khi k tăng; tại k=3, Silhouette Score = 0.4462. Vì bộ dữ liệu Penguins có 3 loài thực tế (Adelie, Chinstrap, Gentoo), nhóm chọn k=3 để mô hình vừa có ý nghĩa thống kê tốt (Silhouette cao thứ nhì) vừa phản ánh đúng cấu trúc sinh học của dữ liệu.

Bước 3: Xây dựng mô hình K-Means với k=3

```
kmeans = KMeans(n_clusters=3, init="k-means++",
                n_init=10, random_state=42).fit(X)
df["kmeans_cluster"] = kmeans.labels_
print(silhouette_score(X, kmeans.labels_)) # 0.4462
```



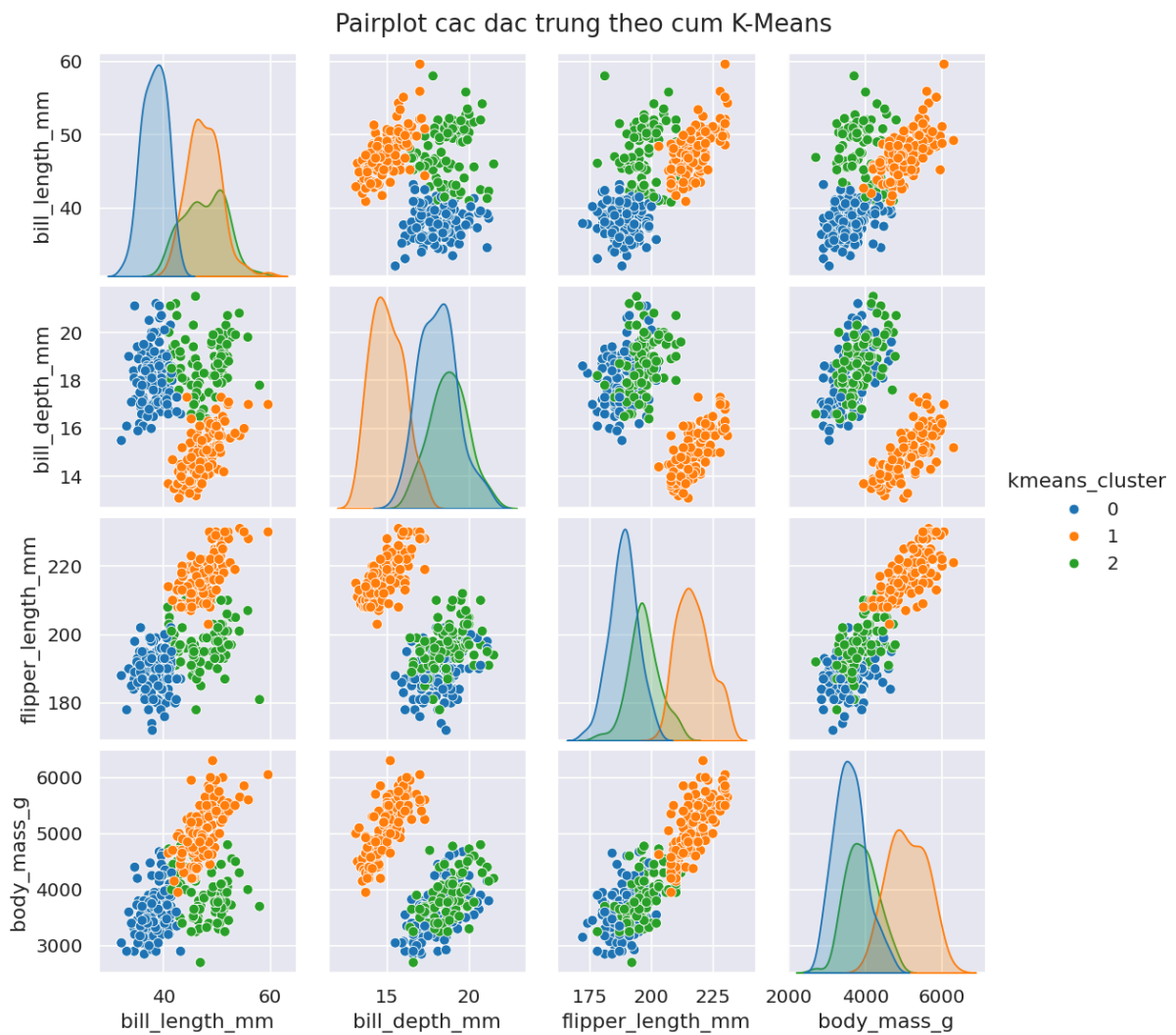
Hình 2 - So sánh cụm K-Means (trái) với loài penguin thực tế (phải)

Bảng đối chiếu giữa cụm K-Means và loài thực tế:

Cụm K-Means	Adelie	Chinstrap	Gentoo
0	124	5	0
1	0	0	119
2	22	63	0

Hồ sơ đặc trưng trung bình của từng cụm:

Cụm	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	so_luong
0.0	38.3	18.1	188.6	3593.8	129.0
1.0	47.6	15.0	217.2	5092.4	119.0
2.0	47.7	18.7	196.9	3898.2	85.0



Hình 3 - Pairplot các đặc trưng theo cụm K-Means

Kết luận: Kết quả phân cụm K-Means ($k=3$) khớp khá tốt với loài thực tế - Cụm 1 tương ứng gần như hoàn toàn với Gentoo (119/119), Cụm 0 chủ yếu là Adelie (124/129), còn Cụm 2 là hỗn hợp giữa Adelie và Chinstrap (do hai loài này có kích thước cơ thể khá giống nhau, chỉ khác biệt rõ ở chiều dài/độ sâu mỏ). Điều này cho thấy K-Means nắm bắt tốt cấu trúc phân nhóm tự nhiên dựa trên đặc trưng hình thái, nhưng vẫn có nhầm lẫn ở các loài có đặc trưng chồng lấn nhau.

3.1.4. Bài tập thực hành 2: Phân cụm K-Means trên tập dữ liệu Online Retail

Đề bài: Xây dựng mô hình phân cụm K-Means trên tập dữ liệu mua sắm/bán lẻ trực tuyến. Nhóm sử dụng bộ dữ liệu Online Retail (giao dịch của một công ty bán lẻ trực tuyến tại Anh, giai đoạn 2010-2011) để thực hiện phân khúc khách hàng (Customer Segmentation) theo mô hình RFM (Recency - Frequency - Monetary).

Dữ liệu lấy tại: <https://www.kaggle.com/datasets/hellbuoy/online-retail-customer-clustering>

Bước 1: Làm sạch dữ liệu giao dịch

```
df = pd.read_csv("OnlineRetail.csv", encoding="latin1")
df = df.dropna(subset=["CustomerID"])
df = df[~df["InvoiceNo"].astype(str).startswith("C")] # bỏ đơn hủy
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
df["TotalPrice"] = df["Quantity"] * df["UnitPrice"]
```

Dữ liệu gốc gồm các cột: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country. Sau khi loại bỏ các dòng thiếu mã khách hàng, các đơn hàng bị hủy (mã hóa đơn bắt đầu bằng "C") và các giá trị số lượng/đơn giá không hợp lệ, còn lại tập dữ liệu giao dịch sạch dùng để tính chỉ số RFM cho từng khách hàng.

Bước 2: Xây dựng bảng RFM cho từng khách hàng

```
import datetime as dt

snapshot_date = df["InvoiceDate"].max() + dt.timedelta(days=1)
rfm = df.groupby("CustomerID").agg(
    Recency=("InvoiceDate", lambda x: (snapshot_date - x.max()).days),
    Frequency=("InvoiceNo", "nunique"),
    Monetary=("TotalPrice", "sum")
).reset_index()
```

Recency: số ngày kể từ lần mua gần nhất. Frequency: số hóa đơn (đơn hàng) khác nhau. Monetary: tổng giá trị đã chi tiêu. Nhóm loại bỏ 1% khách hàng có Monetary cực đoan (top/bottom) để tránh các cụm bị lệch bởi ngoại lệ. Sau xử lý, tập dữ liệu còn 1.115 khách hàng.

Bước 3: Chuẩn hóa và xác định số cụm tối ưu

```
import numpy as np
from sklearn.preprocessing import StandardScaler

# Log-transform vì Frequency/Monetary lệch phải mạnh
X_log = np.log1p(rfm[["Recency", "Frequency", "Monetary"]])
X = StandardScaler().fit_transform(X_log)

inertias, scores = [], []
for k in range(2, 10):
    km = KMeans(n_clusters=k, init="k-means++",
                n_init=10, random_state=42).fit(X)
    inertias.append(km.inertia_)
    scores.append(silhouette_score(X, km.labels_))
```



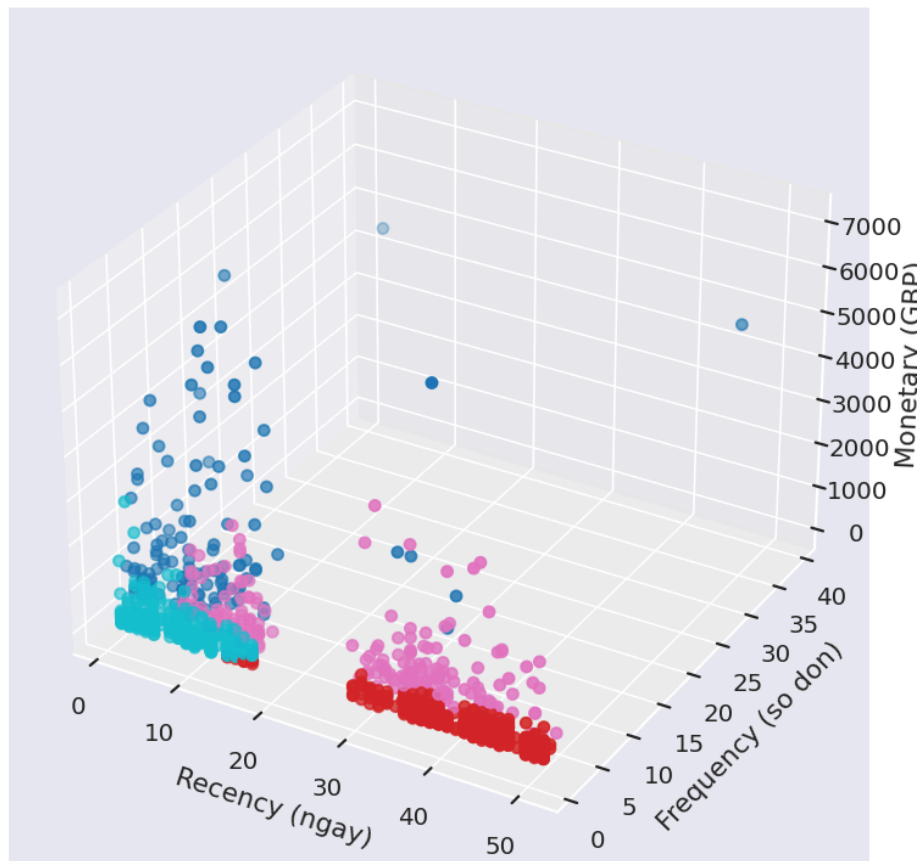
Hình 4 - Biểu đồ Elbow và Silhouette Score cho dữ liệu RFM

Elbow Method cho thấy điểm gãy rõ nhất ở khoảng $k=3-4$. Nhóm chọn $k=4$ để có đủ độ chi tiết phân khúc phục vụ mục tiêu kinh doanh (ví dụ: phân biệt nhóm khách VIP với nhóm khách sắp rời bỏ), với Silhouette Score = 0.3989 - mức chấp nhận được cho dữ liệu hành vi khách hàng vốn có nhiều nhiễu và không tách biệt tuyệt đối.

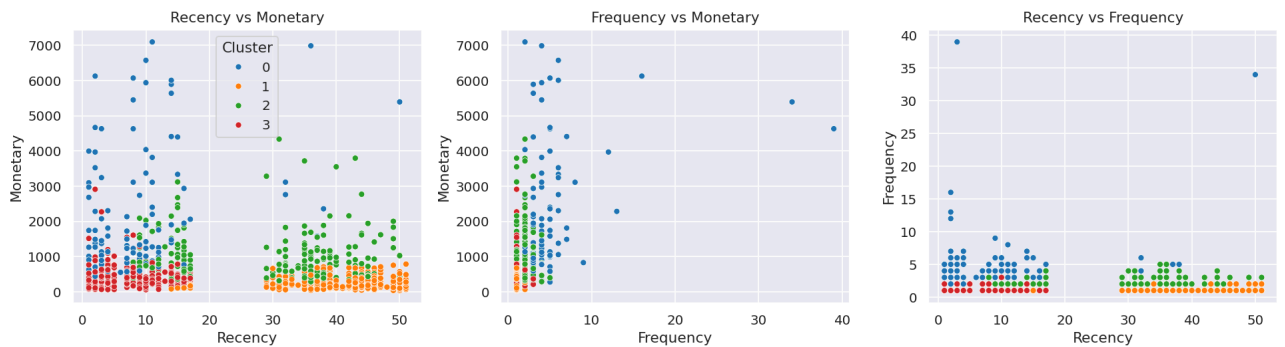
Bước 4: Xây dựng mô hình và trực quan hóa

```
kmeans = KMeans(n_clusters=4, init="k-means++",
                n_init=10, random_state=42).fit(X)
rfm["Cluster"] = kmeans.labels_
```

Phân cụm khách hàng theo RFM (K-Means, k=4)



Hình 5 - Phân cụm khách hàng theo RFM trong không gian 3D



Hình 6 - Phân cụm khách hàng theo từng cặp đặc trưng RFM

Hồ sơ (profile) trung bình của từng cụm khách hàng (đã sắp xếp theo Monetary giảm dần):

Cụm	Recency	Frequency	Monetary	So khách hàng	Tỷ lệ %
0.0	8.9	4.8	2138.0	116.0	10.4
2.0	30.0	2.1	991.3	222.0	19.9
3.0	8.0	1.2	398.2	277.0	24.8
1.0	41.7	1.0	263.7	500.0	44.8

Diễn giải ý nghĩa kinh doanh của các cụm:

- Cụm chi tiêu cao, mua thường xuyên, mới mua gần đây (Recency thấp, Frequency và Monetary cao): đây là nhóm khách hàng VIP/trung thành, cần chương trình chăm sóc và giữ chân đặc biệt.
- Cụm mua khá thường xuyên, giá trị trung bình: nhóm khách hàng tiềm năng, có thể đẩy mạnh upsell/cross-sell.
- Cụm mới mua gần đây nhưng tần suất và giá trị còn thấp: nhóm khách hàng mới, cần khuyến khích mua lại lần hai.
- Cụm Recency cao (lâu chưa quay lại), Frequency và Monetary thấp: nhóm khách hàng có nguy cơ rời bỏ (churn), chiếm tỷ trọng lớn nhất, cần chiến dịch tái kích hoạt (win-back campaign).

3.2. GIẢI THUẬT PHÂN CỤM ĐA CẤP

3.2.1. Ôn tập lý thuyết

1. Giải thuật phân cụm đa cấp hoạt động như thế nào? Giải thích sự khác biệt giữa phân cụm đa cấp hợp nhất (agglomerative) và phân tách (divisive).

Phân cụm đa cấp (hierarchical clustering) xây dựng một cấu trúc cây phân cấp (dendrogram) thể hiện mối quan hệ lồng nhau giữa các cụm, thay vì chỉ tạo ra một phân hoạch phẳng như K-Means. Có hai chiến lược:

- Agglomerative (hợp nhất, bottom-up): bắt đầu với mỗi điểm dữ liệu là một cụm riêng lẻ, sau đó lặp lại việc gộp hai cụm gần nhau nhất thành một cụm lớn hơn, cho đến khi chỉ còn một cụm duy nhất chứa toàn bộ dữ liệu. Đây là cách tiếp cận phổ biến nhất, được cài đặt trong Scikit-learn (AgglomerativeClustering) và SciPy (linkage).

- Divisive (phân tách, top-down): bắt đầu với toàn bộ dữ liệu là một cụm duy nhất, sau đó lặp lại việc chia cụm thành các cụm con cho đến khi mỗi điểm là một cụm riêng. Cách tiếp cận này ít phổ biến hơn vì độ phức tạp tính toán cao hơn nhiều so với agglomerative.

2. Các phương pháp liên kết (linkage) như single linkage, complete linkage, average linkage, và Ward's method khác nhau ra sao? Khi nào nên sử dụng từng loại?

Linkage quyết định cách đo khoảng cách giữa hai cụm khi xét gộp cụm:

- Single linkage: khoảng cách giữa hai cụm là khoảng cách nhỏ nhất giữa một cặp điểm bất kỳ thuộc hai cụm. Có xu hướng tạo ra hiệu ứng chuỗi (chaining effect), phù hợp phát hiện cụm có hình dạng kéo dài, không lồi, nhưng dễ nhạy cảm với nhiễu/outlier.

- Complete linkage: khoảng cách giữa hai cụm là khoảng cách lớn nhất giữa một cặp điểm. Tạo ra các cụm chặt, compact hơn, ít bị chaining nhưng có thể phá vỡ các cụm lớn thành nhiều cụm nhỏ.

- Average linkage: khoảng cách giữa hai cụm là trung bình khoảng cách giữa tất cả các cặp điểm thuộc hai cụm; là phương án cân bằng giữa single và complete linkage.

- Ward's method: gộp hai cụm sao cho mức tăng của tổng phương sai nội cụm (tương tự WCSS trong K-Means) là nhỏ nhất. Thường cho kết quả cụm có kích thước đồng đều, chặt, và là lựa chọn mặc định phổ biến khi dữ liệu có dạng hình cầu tương tự giả định của K-Means (yêu cầu dùng khoảng cách Euclidean).

Lựa chọn: dùng single linkage khi cần phát hiện cụm hình dạng bất thường; dùng complete/average khi muốn cụm chặt, cân đối; dùng Ward khi muốn kết quả gần giống K-Means và dữ liệu tương đối sạch, không quá nhiều outlier.

3. Dendrogram trong phân cụm đa cấp là gì? Làm thế nào để sử dụng nó để chọn số lượng cụm?

Dendrogram là biểu đồ hình cây thể hiện toàn bộ quá trình gộp cụm (hoặc tách cụm) theo thứ tự khoảng cách tăng dần: trục ngang (hoặc dọc tùy cách vẽ) là các điểm/cụm dữ liệu, trục còn lại thể hiện khoảng cách (mức độ khác biệt) tại thời điểm hai cụm được gộp lại.

Để chọn số cụm, ta "cắt" dendrogram bằng một đường ngang tại một mức khoảng cách nhất định: số lần đường cắt này giao với các nhánh dọc chính là số cụm thu được. Thông thường chọn mức cắt ngay dưới nhánh dọc dài nhất (khoảng cách nhảy vọt lớn nhất giữa hai lần gộp liên tiếp), vì đó là dấu hiệu các cụm ở mức đó khác biệt rõ rệt nhất so với việc gộp thêm.

4. Phân cụm đa cấp có thể áp dụng cho dữ liệu phi số (non-numeric data) như thế nào? Hãy giải thích.

Phân cụm đa cấp có thể áp dụng cho dữ liệu phi số bằng cách thay thế độ đo khoảng cách Euclidean bằng một độ đo phù hợp với kiểu dữ liệu, ví dụ: khoảng cách Hamming hoặc hệ số tương đồng Jaccard cho dữ liệu nhị phân/phân loại, khoảng cách Gower cho dữ liệu hỗn hợp (vừa số vừa phân loại), hoặc khoảng cách chỉnh sửa (edit distance) cho dữ liệu chuỗi văn bản.

Sau khi có ma trận khoảng cách (distance matrix) phù hợp, các bước gộp cụm/linkage vẫn thực hiện như bình thường - đây là điểm khác biệt so với K-Means (vốn yêu cầu tính "trung bình" của các điểm nên khó áp dụng trực tiếp cho dữ liệu phân loại).

5. Viết đoạn code mẫu bằng Python (sử dụng Scikit-learn) để triển khai phân cụm đa cấp hợp nhất (agglomerative clustering). Mô tả các bước thực hiện.

Các bước: (1) chuẩn hóa dữ liệu, (2) khởi tạo mô hình AgglomerativeClustering với số cụm và phương pháp linkage mong muốn, (3) huấn luyện và lấy nhãn cụm.

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

ac = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = ac.fit_predict(X_scaled)
df["cluster"] = labels
```

6. Làm thế nào để vẽ dendrogram trong Python sử dụng thư viện như scipy hoặc matplotlib? Chia sẻ đoạn code mẫu.

Sử dụng hàm linkage() của SciPy để tính ma trận liên kết, sau đó dùng dendrogram() để vẽ.

```
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

Z = linkage(X_scaled, method="ward", metric="euclidean")
plt.figure(figsize=(10, 5))
dendrogram(Z, truncate_mode="lastp", p=30, leaf_rotation=90.)
plt.title("Dendrogram")
plt.xlabel("Kích thước cụm / màu")
plt.ylabel("Khoảng cách")
plt.show()
```

7. Các lớp trong gói Scipy hỗ trợ phân cụm đa cấp? So sánh giữa cách tiếp cận Scikit-learn và Scipy.

Module scipy.cluster.hierarchy cung cấp các hàm chính: linkage() (tính ma trận liên kết theo các phương pháp single/complete/average/ward/centroid...), dendrogram() (vẽ biểu đồ cây), và fcluster() (cắt dendrogram tại một ngưỡng để lấy nhãn cụm cụ thể).

So sánh hai cách tiếp cận:

- Scikit-learn (AgglomerativeClustering): giao diện đơn giản, thống nhất với các thuật toán phân cụm khác (fit_predict), dễ tích hợp vào pipeline machine learning, nhưng không trực tiếp cung cấp công cụ vẽ dendrogram và ít linh hoạt hơn trong việc kiểm soát ma trận liên kết.
- SciPy (linkage/dendrogram/fcluster): cung cấp đầy đủ ma trận liên kết trung gian, cho phép vẽ dendrogram chi tiết và cắt cụm theo nhiều tiêu chí khác nhau (distance, maxclust...), phù hợp khi cần phân tích/khám phá cấu trúc phân cấp một cách trực quan trước khi quyết định số cụm.

Trong thực hành, thường dùng SciPy để khám phá dữ liệu qua dendrogram, sau đó dùng Scikit-learn để huấn luyện mô hình cuối cùng trong pipeline.

3.2.3. Bài tập thực hành 1: Phân cụm đa cấp trên tập dữ liệu Penguins

Đề bài: Xây dựng mô hình phân cụm đa cấp trên tập dữ liệu chim cánh cụt (Penguins), sử dụng cùng 4 đặc trưng số đã chuẩn hóa như ở phần K-Means để có thể so sánh trực tiếp kết quả giữa hai giải thuật.

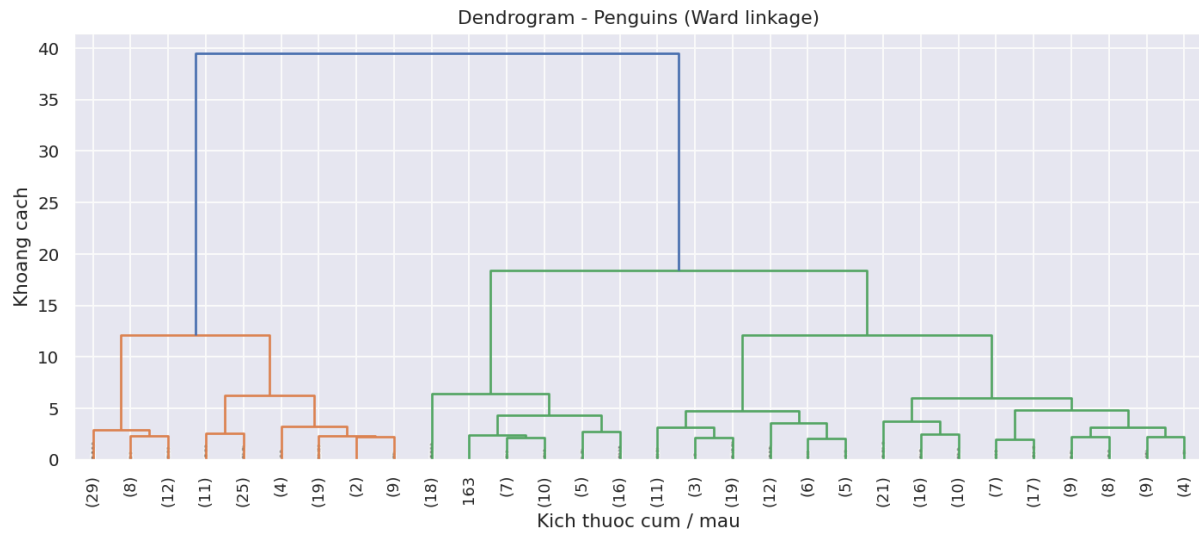
Dữ liệu lấy tại:

<https://www.kaggle.com/code/youssefaboelwafa/clustering-penguins-species-k-means-clustering>

Bước 1: Vẽ dendrogram với Ward linkage

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
```

```
Z = linkage(X, method="ward", metric="euclidean")
dendrogram(Z, truncate_mode="lastp", p=30, leaf_rotation=90.)
```

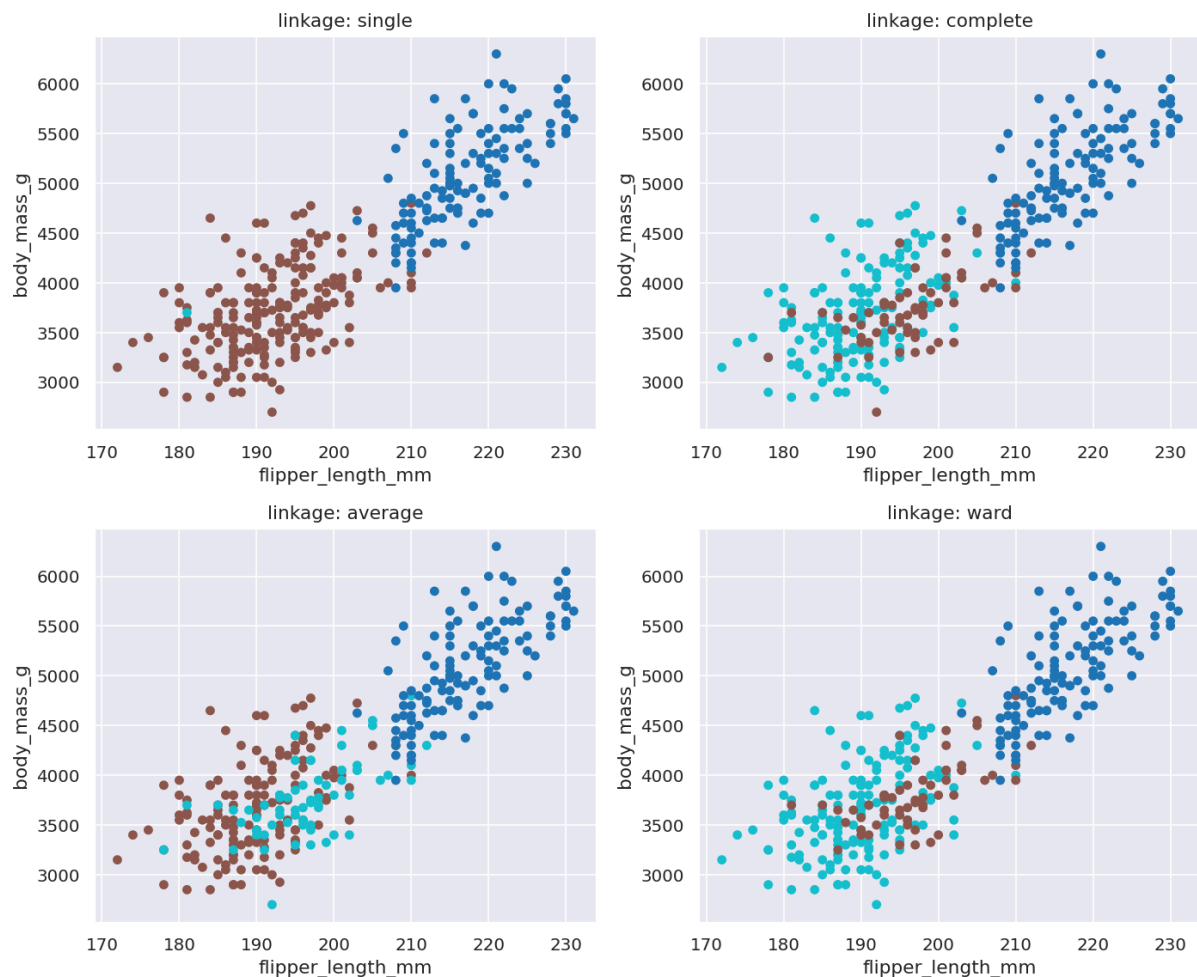


Hình 7 - Dendrogram tập dữ liệu Penguins (Ward linkage)

Quan sát dendrogram cho thấy 3 nhánh chính tách biệt rõ rệt ở mức khoảng cách lớn, phù hợp với giả thuyết dữ liệu có 3 nhóm tự nhiên (tương ứng 3 loài penguin).

Bước 2: So sánh các phương pháp linkage

```
methods = ["single", "complete", "average", "ward"]
for method in methods:
    Zm = linkage(X, method=method, metric="euclidean")
    clusters = fcluster(Zm, 3, criterion="maxclust")
```



Hình 8 - So sánh kết quả phân cụm giữa các phương pháp linkage ($k=3$)

Single linkage cho hiệu ứng chuỗi rõ rệt (gần như dồn phần lớn điểm vào một cụm lớn do nhạy cảm với các điểm nằm giữa hai nhóm), trong khi complete, average và đặc biệt là Ward linkage cho ra các cụm cân đối và khớp tốt với cấu trúc dữ liệu hơn. Nhóm chọn Ward linkage cho mô hình cuối cùng.

Bước 3: Xây dựng mô hình cuối cùng và trực quan hóa

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

ac = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = ac.fit_predict(X)
df["hc_cluster"] = labels
print(silhouette_score(X, labels)) # 0.4521
```



Hình 9 - So sánh cụm phân cụm đa cấp (trái) với loài penguin thực tế (phải)

Bảng đối chiếu giữa cụm phân cụm đa cấp và loài thực tế:

Cụm (Ward)	Adelie	Chinstrap	Gentoo
0	0	0	119
1	146	11	0
2	0	57	0

Bước 4: So sánh với K-Means bằng Silhouette Score

Phương pháp	Silhouette Score
K-Means	0.4462
Agglomerative (Ward)	0.4521

Kết quả cho thấy trên tập dữ liệu Penguins, Agglomerative Clustering (Ward, k=3) đạt Silhouette Score (0.4521) nhỉnh hơn một chút so với K-Means (0.4462), và cách phân nhóm cũng rất tương đồng: cụm Gentoo được tách hoàn toàn chính xác ở cả hai phương pháp, trong khi Adelie và Chinstrap vẫn có sự chồng lấn nhất định do đặc trưng hình thái gần giống nhau.

3.2.4. Bài tập thực hành 2: Phân cụm đa cấp trên tập dữ liệu Online Retail

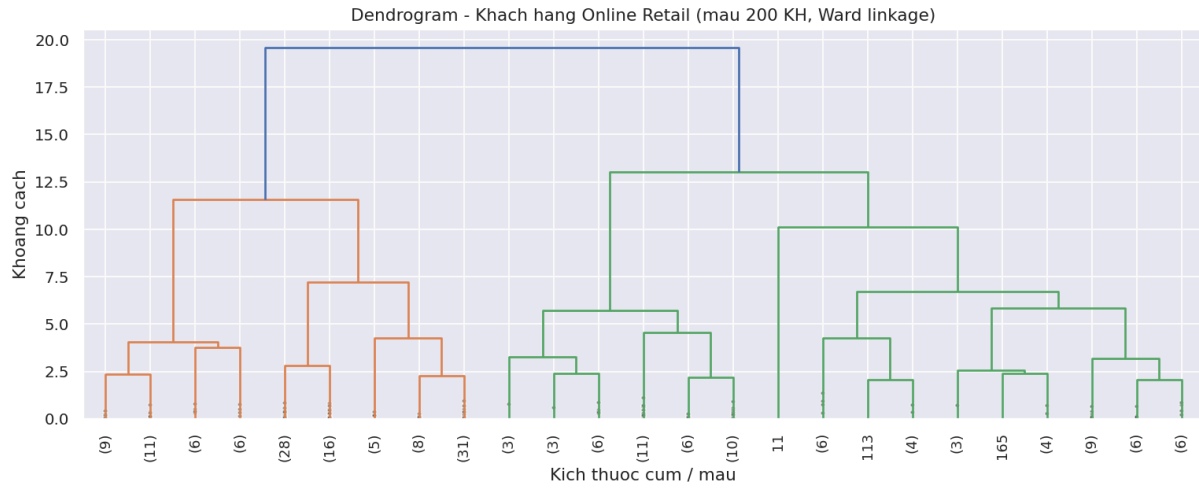
Đề bài: Xây dựng mô hình phân cụm đa cấp trên tập dữ liệu mua sắm/bán lẻ trực tuyến, sử dụng lại bảng RFM (Recency - Frequency - Monetary) đã xây dựng ở phần K-Means (1.115 khách hàng) để so sánh trực tiếp kết quả giữa hai giải thuật.

Dữ liệu lấy tại: <https://www.kaggle.com/datasets/hellbuoy/online-retail-customer-clustering>

Bước 1: Vẽ dendrogram trên mẫu khách hàng

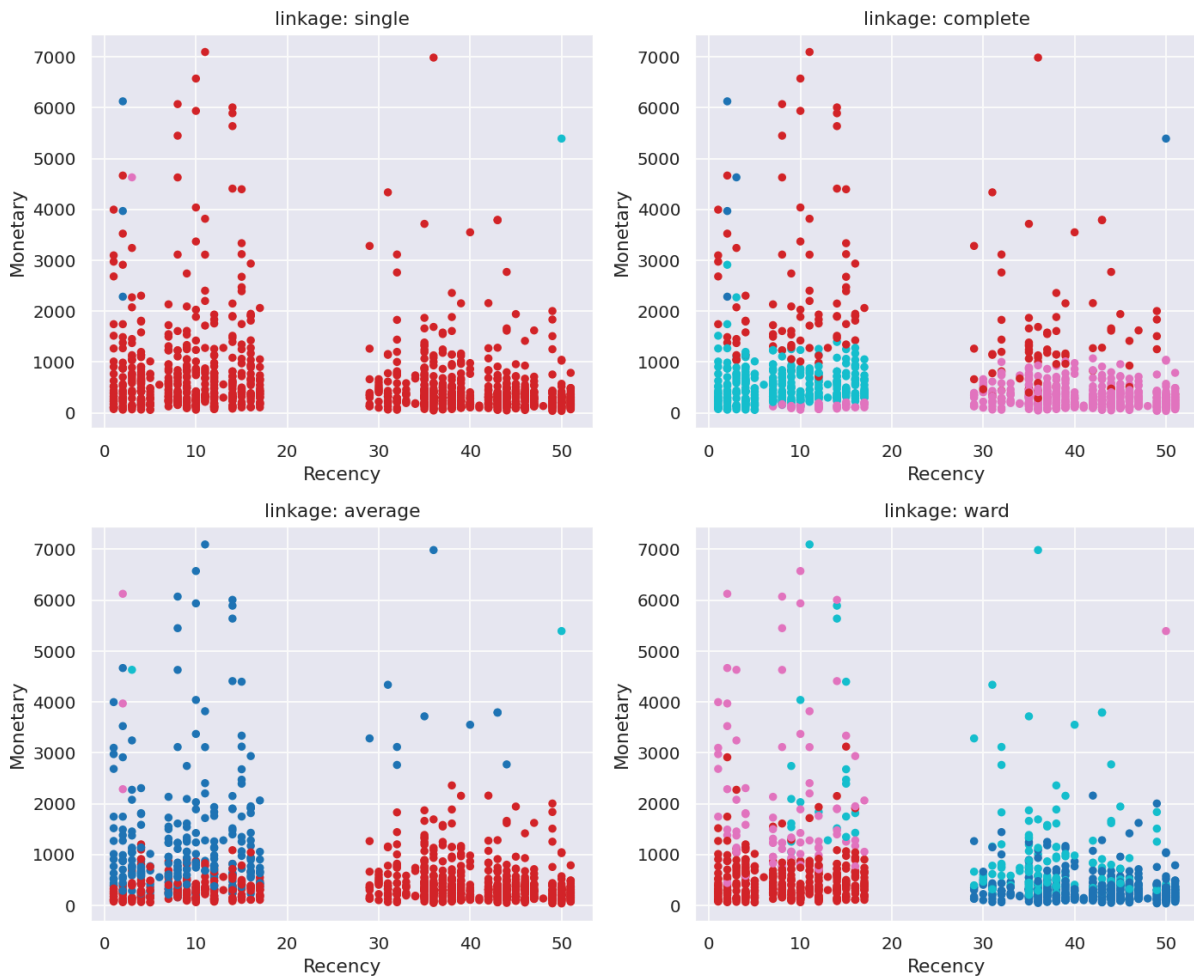
Vì tập dữ liệu có 1.115 khách hàng (khá lớn để vẽ dendrogram đầy đủ và dễ đọc), nhóm lấy mẫu ngẫu nhiên 200 khách hàng để minh họa cấu trúc phân cấp.

```
sample_idx = np.random.RandomState(42).choice(
    len(X), size=200, replace=False)
Z = linkage(X[sample_idx], method="ward", metric="euclidean")
dendrogram(Z, truncate_mode="lastp", p=25, leaf_rotation=90.)
```



Hình 10 - Dendrogram (mẫu 200 khách hàng, Ward linkage)

Bước 2: So sánh các phương pháp linkage ($k=4$)

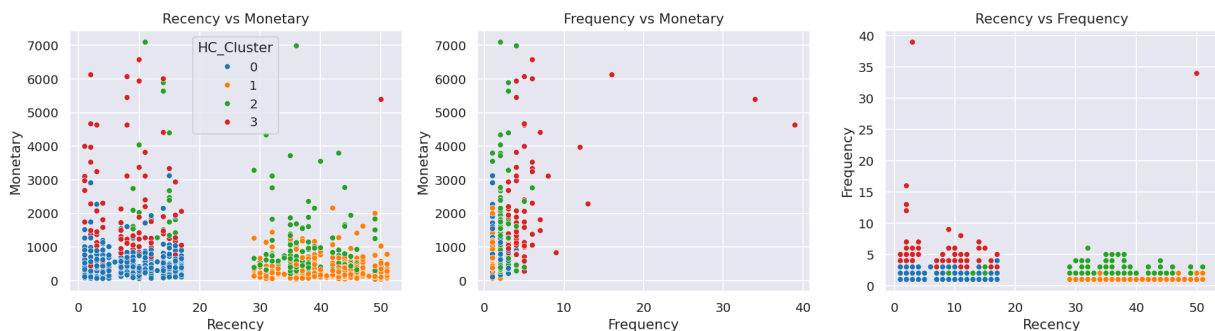


Hình 11 - So sánh kết quả phân cụm khách hàng giữa các phương pháp linkage

Tương tự tập Penguins, single và complete linkage cho kết quả mất cân đối (phần lớn khách hàng dồn vào 1-2 cụm lớn), trong khi Ward linkage cho ra 4 cụm có kích thước hợp lý hơn và phản ánh tốt hơn sự khác biệt về hành vi mua sắm.

Bước 3: Xây dựng mô hình cuối cùng (Ward, $k=4$) và trực quan hóa

```
ac = AgglomerativeClustering(n_clusters=4, linkage="ward")
rfm["HC_Cluster"] = ac.fit_predict(X)
print(silhouette_score(X, rfm["HC_Cluster"])) # 0.3755
```



Hình 12 - Phân cụm khách hàng theo phân cụm đa cấp (Ward, $k=4$)

Hồ sơ trung bình của từng cụm khách hàng theo phân cụm đa cấp:

Cụm	Recency	Frequency	Monetary	So_khach_hang	Ty_le %
3.0	8.8	5.4	2139.2	88.0	7.9
2.0	34.5	2.4	1177.8	163.0	14.6
0.0	8.9	1.5	476.8	363.0	32.6
1.0	42.2	1.0	313.3	501.0	44.9

Bước 4: So sánh với K-Means bằng Silhouette Score

Phương pháp	Silhouette Score
K-Means ($k=4$)	0.3989
Agglomerative Ward ($k=4$)	0.3755

Trên tập dữ liệu RFM Online Retail, K-Means (Silhouette = 0.3989) cho kết quả tách cụm tốt hơn một chút so với Agglomerative Ward (Silhouette = 0.3755). Điều này phù hợp lý thuyết vì dữ liệu RFM sau log-transform có phân bố tương đối gần dạng cầu (spherical), là điều kiện thuận lợi cho K-Means; đồng thời K-Means tối ưu trực tiếp minh bạch dựa trên khoảng cách tới tâm cụm nên phù hợp hơn với bài toán phân khúc khách hàng có số lượng mẫu lớn. Tuy vậy, cả hai phương pháp đều cho ra cấu trúc 4 nhóm khách hàng tương đồng về mặt ý nghĩa kinh doanh (khách VIP, khách tiềm năng, khách mới, khách có nguy cơ rời bỏ).

D. TÓM TẮT

Báo cáo đã trình bày và thực hành hai giải thuật phân cụm cơ bản là K-Means và phân cụm đa cấp (Hierarchical/Agglomerative Clustering) trên hai bộ dữ liệu thực tế: Penguins (dữ liệu hình thái sinh học) và Online Retail (dữ liệu giao dịch bán lẻ, được chuyển hóa thành chỉ số RFM để phân khúc khách hàng).

Kết quả cho thấy trên tập Penguins - nơi các cụm có xu hướng hình cầu và số lượng mẫu vừa phải - phân cụm đa cấp (Ward linkage) đạt Silhouette Score nhỉnh hơn K-Means một chút, trong khi trên tập RFM Online Retail có quy mô lớn hơn, K-Means lại cho kết quả tốt hơn. Cả hai giải thuật đều là công cụ hữu ích để khám phá cấu trúc nhóm tiềm ẩn trong dữ liệu không có nhãn, tuy nhiên việc lựa chọn giải thuật và số cụm phù hợp cần dựa trên đặc điểm phân bố dữ liệu, quy mô tập dữ liệu, và mục tiêu ứng dụng thực tế (ví dụ mục tiêu kinh doanh trong phân khúc khách hàng).

Việc thực hành giải quyết các bài tập trên hai tập dữ liệu khác nhau giúp hiểu rõ hơn các khái niệm lý thuyết đã học (Elbow Method, Silhouette Score, các phương pháp linkage, dendrogram...) và áp dụng tốt vào việc giải quyết các bài toán phân cụm trong thực tế.